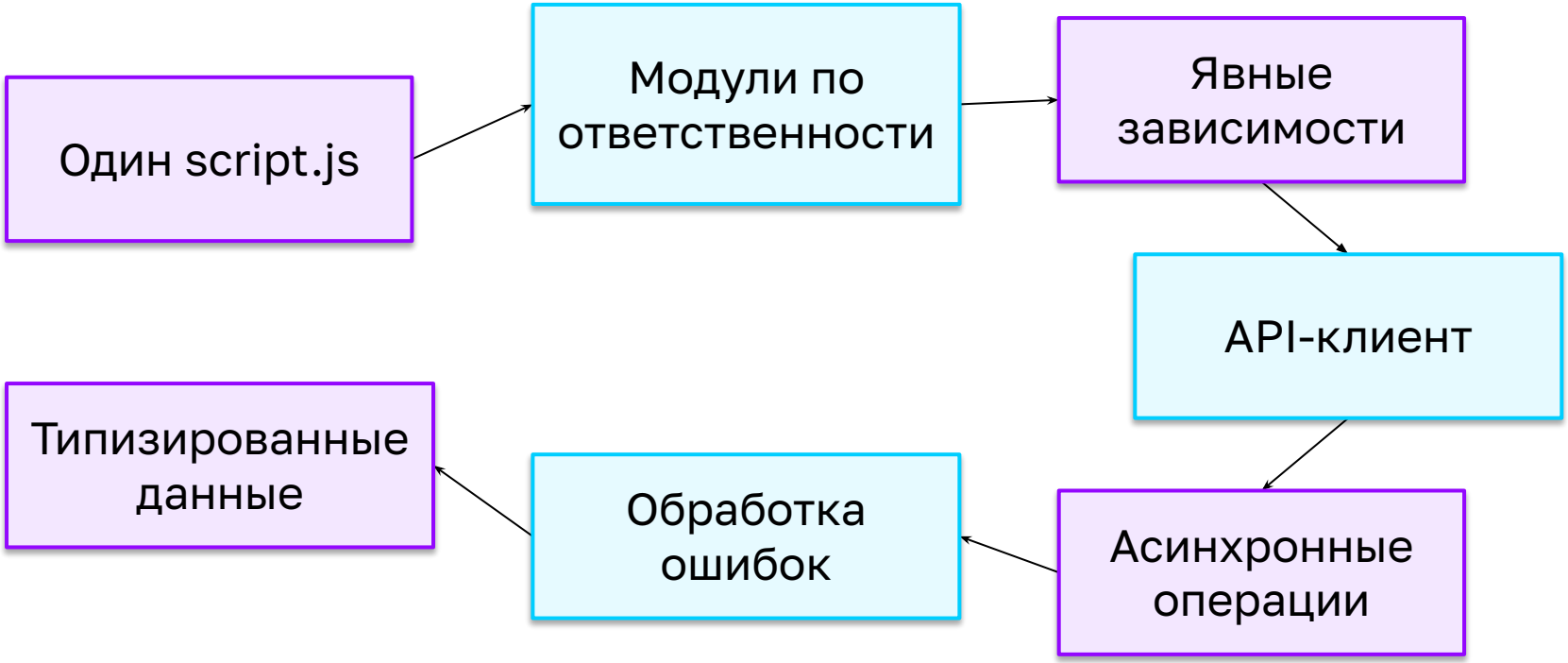
ИТМО

Современный JavaScript, TypeScript и браузерные API

От скрипта к приложению

ИТМО



ES-модули

ИТМО

Определение

- ES-модуль - стандарт деления JavaScript-кода
- У файла своя изолированная область видимости
- Зависимости перечисляются явно

Что это даёт

- Нет загрязнения window
- Функции проще переиспользовать
- Модули легче тестировать и менять
- import() загружает код по требованию

export и import

ИТМО

```
// config.js
export const API_URL =
"/api";
export default const
API_URL2 = "/api/api"

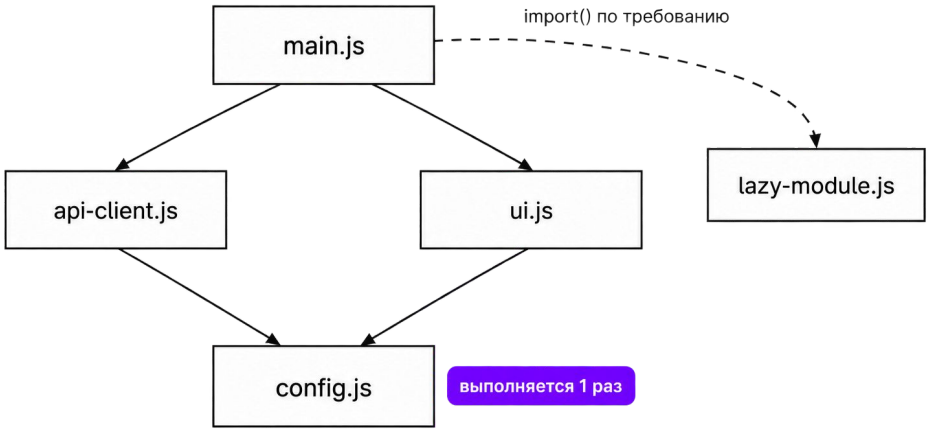
// main.js
import { API_URL } from
"./config.js";
import API_URL2 from
"./config.js";
```

- export открывает только выбранные значения
- import явно фиксирует зависимость
- <script type="module"> подключает точку входа
- Расширение .js в браузере указывают явно

ES-модули

Свойства ES-модуля:

- своя область видимости
- строгий режим (use strict)
- запуск после разбора HTML
- кеширование после выполнения
- import() — код по требованию



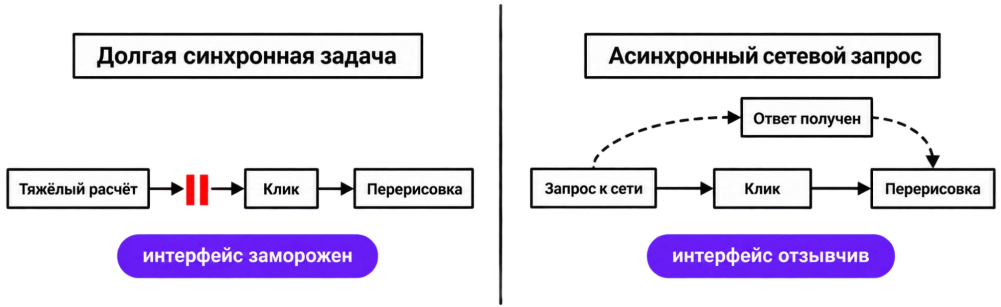
Асинхронность в JS

ИТМО

JS - однопоточный язык. В основном потоке:

- выполняется JavaScript
- обрабатываются события
- меняется DOM
- готовится отображение

Ожидание сети нельзя превращать в блокирующую операцию

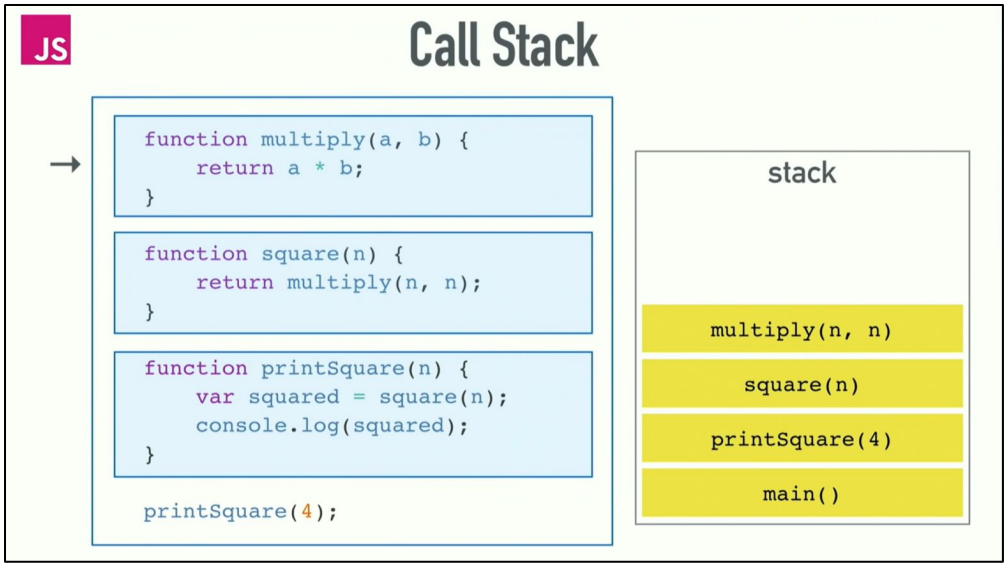


Call stack

ИТМО

Стек вызова - структура, которая отслеживает текущее местонахождение интерпретатора в скрипте.

- Хранит контекст вызываемой функции
- Имеет ограниченный размер
- Выполняет строго одну операцию за раз



Очереди задач

ИТМО

Task (Macrotask) queue

Очередь задач - очередь отложенных асинхронных задач с низким приоритетом.

Среди списка задач:

- setTimeout/setInterval
- DOM события
- операции ввода-вывода (mousemove)
- сетевые запросы

Microtask queue

Очередь микрозадач - очередь отложенных асинхронных задач с высоким приоритетом.

Среди списка задач:

- Обработчики промисов
- Код после await
- Наблюдатель за изменениями DOM (MutationObserver)
- Явно добавленые функции в очередь микрозадач (queueMicrotask)

Event loop

ИТМО

Цикл событий - механизм исполнения асинхронных операция в JS.

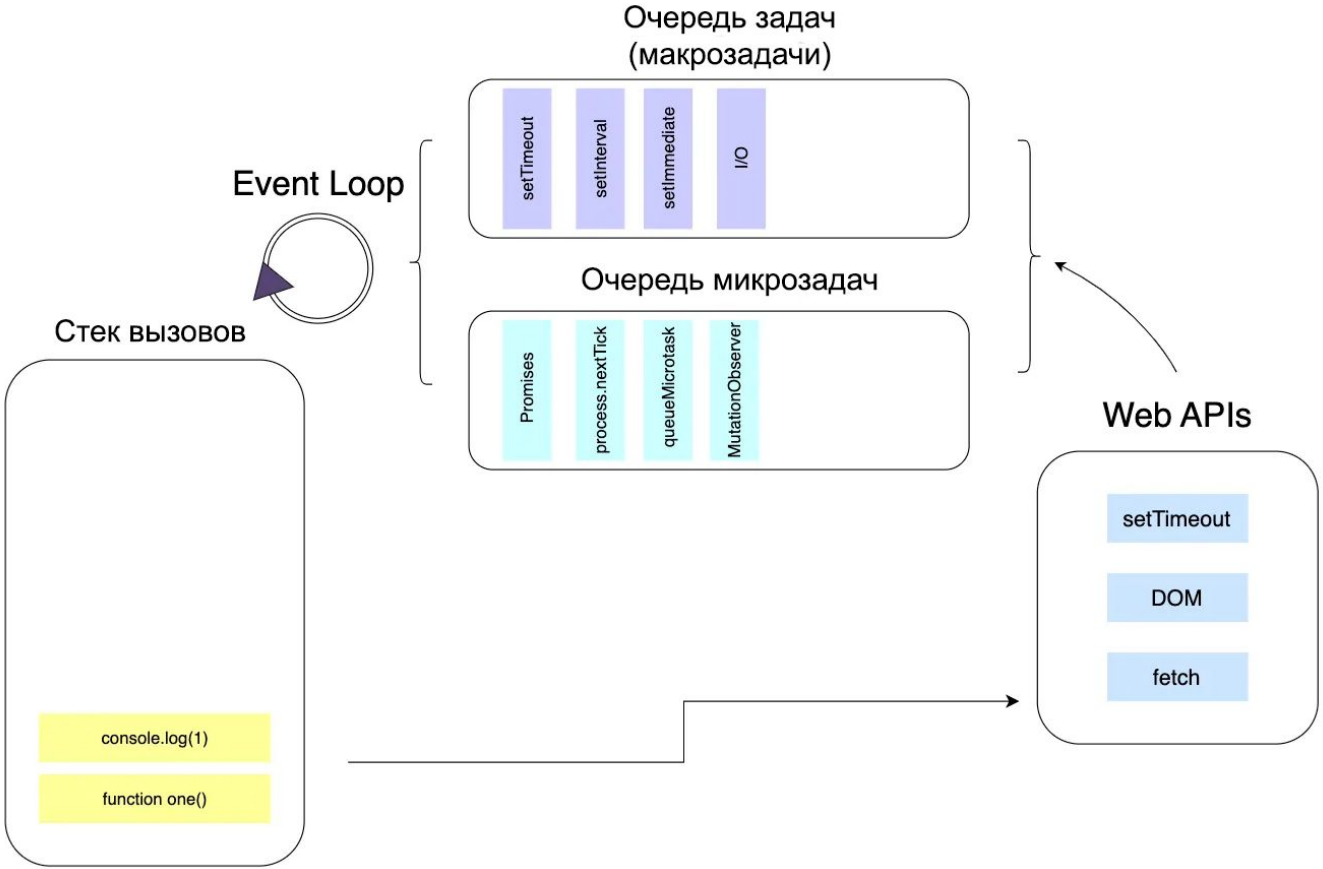
Последовательность:

- Пуст ли стек вызовов?
- Есть ли задачи в очереди микрозадач?
- Есть ли задачи в очереди макрозадач?
- Можем ли обновить интерфейс?

событийный цикл



Event loop

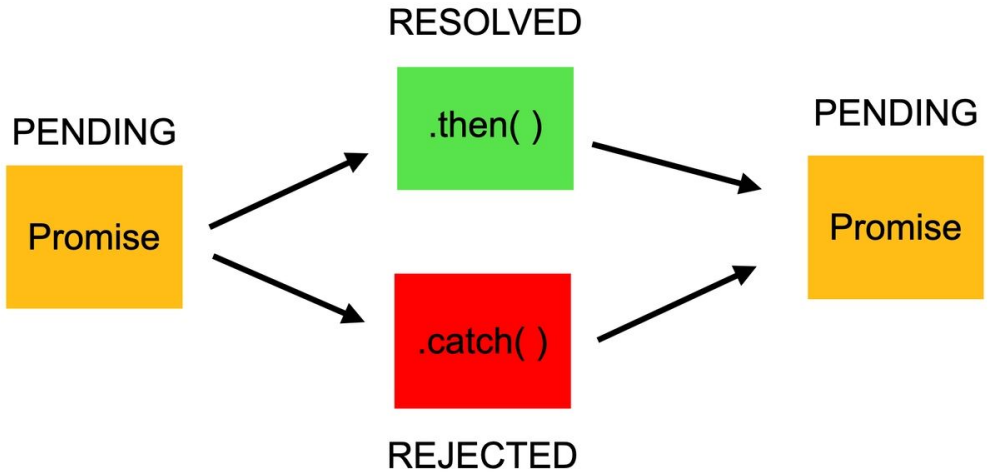

Очередь задач
(макрозадачи)
setTimeout
setInterval
setImmediate
I/O
Event Loop
Очередь микрозадач
Promises
process.nextTick
queueMicrotask
MutationObserver
Стек вызовов
Web APIs
setTimeout
DOM
fetch
console.log(1)
function one()

Promise

ИТМО

Promise - объект-обертка для значения, получаемого в ходе асинхронной операции. Имеет 3 состояния:

- pending (ожидание) - начальное состояние
- fulfilled (успех) - успешное завершение
- rejected (ошибка) - завершение с ошибкой



Promise

ИТМО

```
const isPromiseResolve = true;

const clickPromise = new Promise((resolve, reject) => {
  if (isPromiseResolve) {
    resolve("resolve");
  }

  if (!isPromiseResolve) {
    reject(new Error("reject"));
  }
});

clickPromise.then((result) => {
  console.log(result); //resolve
}).catch((error) => {
  console.error(error.message); //reject
}).finally(() => {
  console.log('finally') //finally
});
```

```
resolve
finally
```

```
const isPromiseResolve = false;

const clickPromise = new Promise((resolve, reject) => {
  if (isPromiseResolve) {
    resolve("resolve");
  }

  if (!isPromiseResolve) {
    reject(new Error("reject"));
  }
});

clickPromise.then((result) => {
  console.log(result); //resolve
}).catch((error) => {
  console.error(error.message); //reject
}).finally(() => {
  console.log('finally') //finally
});
```

```
reject
finally
```

Promise

ИТМО

Цепочка промисов

Перехват ошибок всей цепочки

```
fetch("/api/users/1") //Promise
  .then((response) => {
    if (!response.ok) {
      throw new Error("Не удалось получить данные");
    }

    return response.json(); //тоже Promise
  })
  .then((user) => {
    console.log("User:", user);
  })
  .catch((error) => {
    console.error("Error:", error.message);
  })
  .finally(() => {
    console.log("Request completed");
  });
```

Методы Promise

ИТМО

Promise.all - метод, возвращающий промис, исполняющийся по исполнению всех или отклонению одного из переданных ему промисов.

Есть также:

- Promise.race
- Promise.resolve
- Promise.reject

Гарантируется порядок передаваемых значений, но не завершения!

```
const p1 = Promise.resolve(3);
const p2 = 1337;
const p3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "foo");
});

Promise.all([p1, p2, p3]).then((values) => {
  console.log(values);
});
```

```
[ 3, 1337, 'foo' ]
```

Async/await

async/await - ключевые слова, упрощающие работу с промисами

- async - указывает, что функция всегда возвращает промис
- await - указывает, что интерпретатор ждет окончания выполнения промиса

```
async function f() {

  let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("готово!"), 1000)
  });

  let result = await promise;

  console.log(result); // "готово!"
}

f();
```

```
готово!
```

Async/await

ИТМО

```
async function f() {

  async function getNumberAsync() {
    return 42;
  }

  let result = await getNumberAsync();

  console.log(result); // "42"
}

f();
```

42

```
async function f() {

  function getNumberPromise() {
    return Promise.resolve(42);
  }

  let result = await getNumberPromise();

  console.log(result); // "42"
}

f();
```

42

Обработка ошибок

ИТМО

```
async function loadUser() {
  try {
    const response = await fetch("/api/users/1");

    if (!response.ok) {
      throw new Error("Не удалось загрузить пользователя");
    }

    const user = await response.json();
    console.log("User:", user);
  } catch (error) {
    console.error("Error:", error.message);
  } finally {
    console.log("Request completed");
  }
}

loadUser();
```

AJAX, Fetch API

ИТМО

AJAX

AJAX (Asynchronous JavaScript and XML) - подход, при котором JavaScript обменивается данными с сервером и обновляет часть интерфейса без полной перезагрузки.

Примеры: поиск, фильтры, пагинация, формы, уведомления.

Инструменты

Исторически - XMLHttpRequest.

Сегодня для новых приложений обычно используют Fetch API.

XML в названии не означает, что данные обязаны быть XML: чаще приходит JSON.

XMLHttpRequest

ИТМО



```
const request = new XMLHttpRequest(); //создание объекта

request.open("GET", "/api/users/1"); //конфигурация запроса
request.responseType = "json";

//слушатель на получение ответа
request.addEventListener("load", () => {
  //обработка ответа
  if (request.status >= 200 && request.status < 300) {
    console.log("User:", request.response);
  } else {
    console.error("HTTP error:", request.status);
  }
});

//слушатель на ошибку
request.addEventListener("error", () => {
  console.error("Network error");
});

//отправка запроса
request.send();
```

Fetch API

ИТМО



```
async function loadUser() {
  try {
    const response = await fetch("/api/users/1");

    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }

    const user = await response.json();

    console.log("User:", user);
  } catch (error) {
    console.error("Request error:", error.message);
  }
}

loadUser();
```

Fetch API

ИТМО

Response

Response - класс ответа Fetch API.

Основные свойства:

- response.ok
- response.status
- response.statusText
- response.headers
- response.url

URLSearchParams

URLSearchParams - класс формирования строки поисковых параметров.

```
const params = new URLSearchParams({
   page: "2",
   limit: "20"
 });

 const response = await
fetch(`/api/users?${params}`);
```

Fetch API

ИТМО

same-origin - только на тот же origin;
include - и для cross-origin requests;
omit - не отправлять

Credentials

```
const response = await fetch("/api/users", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        Accept: "application/json",
    },
    body: JSON.stringify(userData),
});
```

```
const response = await fetch(
    "https://api.example.com/profile",
    {
        method: "GET",
        credentials: "include",
        headers: {
            Accept: "application/json",
        },
    }
);
```

Заголовки

JSON

ИТМО

```
const user = {
  name: "Alex",
  age: 25,
};

console.log(JSON.stringify(user));

const json = '{"name":"Alex","age":25}';

console.log(JSON.parse(json).name);
```

```
{"name":"Alex","age":25}
Alex
```

JSON - текстовый формат представления структурированных данных

Методы:

- stringify - JS-объект в JSON
- parse - JSON в JS-объект

Не все типы данных JSON поддерживает напрямую

Валидация получаемого через fetch JSON обязательна!

Web Storage API и Cookies

ИТМО

```
localStorage.setItem("t",
"b");
localStorage.getItem("t");
localStorage.removeItem("t
");
localStorage.clear();
```

localStorage

- Данные без автоматического удаления (даже при перезапуске браузера)
- Подходит для хранения темы/языка интерфейса, настроек и тп

```
sessionStorage.setItem("t"
, "b");
sessionStorage.getItem("t"
);
sessionStorage.removeItem(
"t");
sessionStorage.clear();
```

sessionStorage

- Данные только в рамках текущей вкладки
- Подходит для хранения временной навигации, состояния вкладки, временных фильтров и тп

```
document.cookie = "t=b;
Path=/; Max-Age=3600;
SameSite=Lax";
const cookies =
document.cookie.split(";
");
document.cookie =
  "t=; Path=/; Max-Age=0";
```

cookies

- Данные в рамках серверной сессии или указанного срока
- Подходит для данных серверной сессии

Web Storage API и Cookies

ИТМО

```
localStorage.setItem("t",
"b");
localStorage.getItem("t");
localStorage.removeItem("t
");
localStorage.clear();
```

localStorage

- Данные без автоматического удаления (даже при перезапуске браузера)
- Подходит для хранения темы/языка интерфейса, настроек и тп

```
sessionStorage.setItem("t"
, "b");
sessionStorage.getItem("t"
);
sessionStorage.removeItem(
"t");
sessionStorage.clear();
```

sessionStorage

- Данные только в рамках текущей вкладки
- Подходит для хранения временной навигации, состояния вкладки, временных фильтров и тп

```
document.cookie = "t=b;
Path=/; Max-Age=3600;
SameSite=Lax";
const cookies =
document.cookie.split(";
");
document.cookie =
  "t=; Path=/; Max-Age=0";
```

cookies

- ...или
- ...ых

Автоматически передаются серверу с запросами

Origin

ИТМО

Origin - уникальная комбинация протокола, домена и порта

Origin = http/https + domain + port

Путь к файлу/странице к источнику не относится

```
//разные источники
https://example.com
http://example.com
https://api.example.com
https://example.com:8080
```

Same-Origin Policy и CORS

ИТМО

Same-Origin Policy

Политика одинакового источника - механизм ограничения взаимодействия между ресурсами разных origin. Ограничивает:

- чтение response через fetch/XMLHttpRequest;
- доступ к DOM другой страницы через iframe;
- доступ к данным другого origin в localStorage;
- чтение содержимого некоторых загруженных ресурсов.

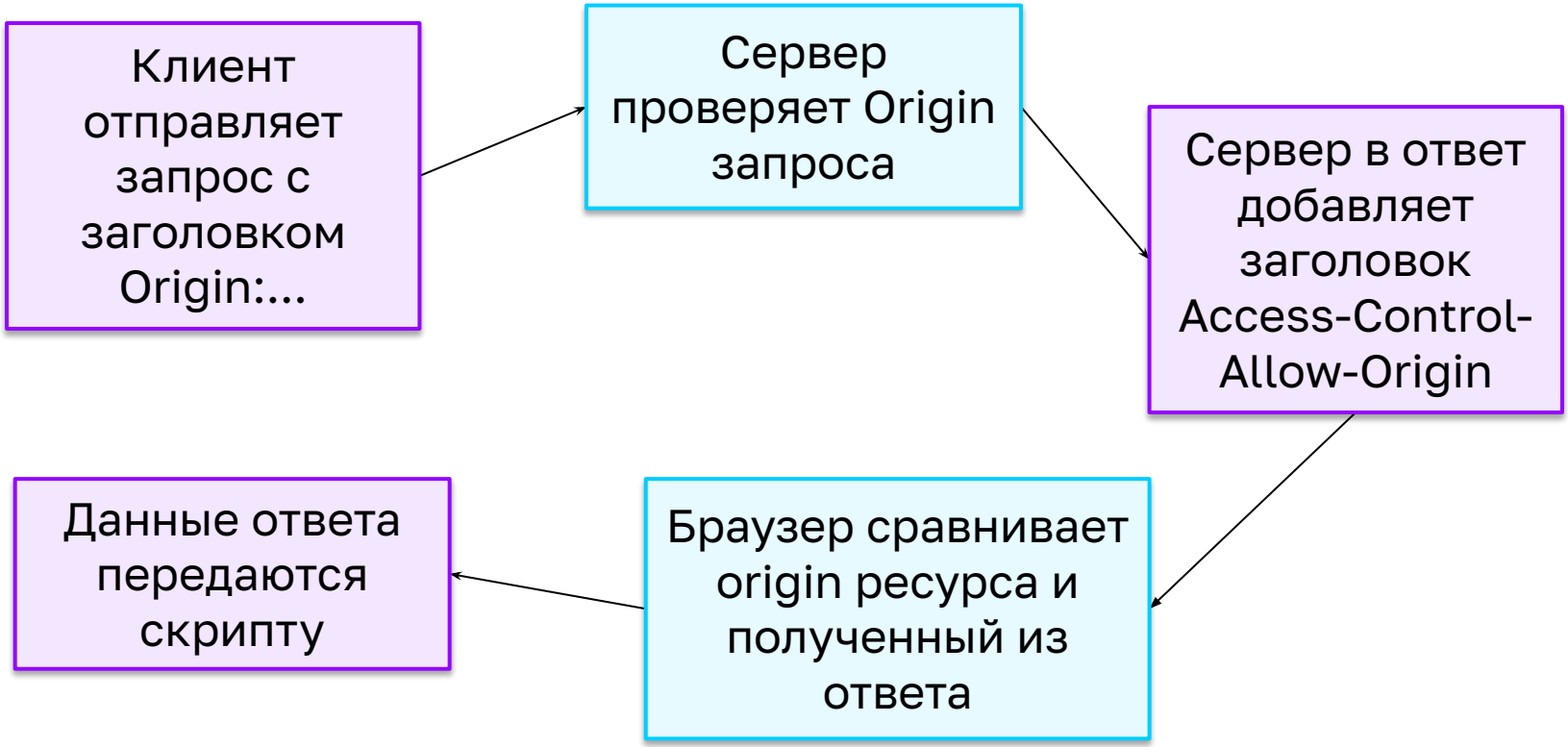
CORS

CORS - механизм разрешения запроса ресурсов одного домена у другого.

Предоставление доступа - через заголовки Origin и Access-Control-Allow-Origin.

Same-Origin Policy и CORS

ИТМО



Simple и Preflight Requests

ИТМО

Simple request

Самый обыкновенный запрос.

Ну правда.

Сравнивает источники, использует GET, POST, HEAD методы

Preflight request

Проверочный запрос перед отправкой основного. Автоматически отправляется браузеров, использует метод OPTIONS. Используется если основной запрос:

- Используется PATCH, PUT, DELETE
- Нестандартные заголовки
- Content-Type: application/json
- Authorization

TypeScript

ИТМО

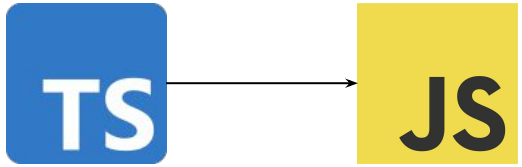
Что это

TypeScript — надмножество JavaScript со статической типизацией.

- Позволяет обнаружить ошибку на этапе написания кода, а не компиляции
- Улучшает читаемость кода
- Позволяет описывать контракты
- Почему то отрицается как технология Евгением Андреевичем

И как это

Не исполняется напрямую браузером, а компилируется в JS после проверки типов.



TS

ИТМО

```
const name: string = "Anna";
const age: number = 25;
const active: boolean =
true;
const roles: string[] =
["user", "admin"];
```

Базовые типы

```
function calculateTotal(
   price: number,
   quantity: number
 ): number {
   return price * quantity;
 }
```

Типизация в функциях

```
type User = {
   readonly id: number;
   name: string;
   email?: string;
   active: boolean;
 };
```

Типизация в объектах

```
interface User {
   id: number;
   name: string;
   email: string;
 }
```

Интерфейсы

```
let data: any;

let data: unknown;

if (typeof data ===
"string") console.log(data);
```

any/unknown

```
type ApiResponse<T> = {
   data: T;
   error: string | null;
 };

type UsersResponse =
ApiResponse<User[]>;
```

Generic-типы

Union/Intersection-типы

ИТМО

Объединение - инструмент указания возможных типов для значения. Использует |.

Пересечение - инструмент указания необходимых типов для значения. Использует &.

Тип можно уточнить через ключевое слово typeof

```ts
type Id = string | number;
let userId: Id;
userId = 10;
userId = "user-10";

type Status = "idle" | "loading" | "success" | "error";
const requestStatus: Status = "loading";

function printId(id: string | number) {
  if (typeof id === "string") {
    //тут тип string
    console.log(id.toUpperCase());
  } else {
    //а тут number
    console.log(id.toFixed(0));
  }
}

type User = {
  id: number;
};

type Permission = {
  canEdit: boolean;
};

//итоговый тип удовлетворяет const admin: Admin = {id: 1, canEdit: true};
type Admin = User & Permission;
```

После уточнения типа присвоить значение другого типа нельзя

Utility Types

ИТМО

```ts
interface Todo {
  title: string;
  description: string;
}

type PartialToDo = Partial<Todo>;
// {title?: string; description?: string;}

type RequiredToDo = Required<PartialToDo>;
// {title: string; description: string;}

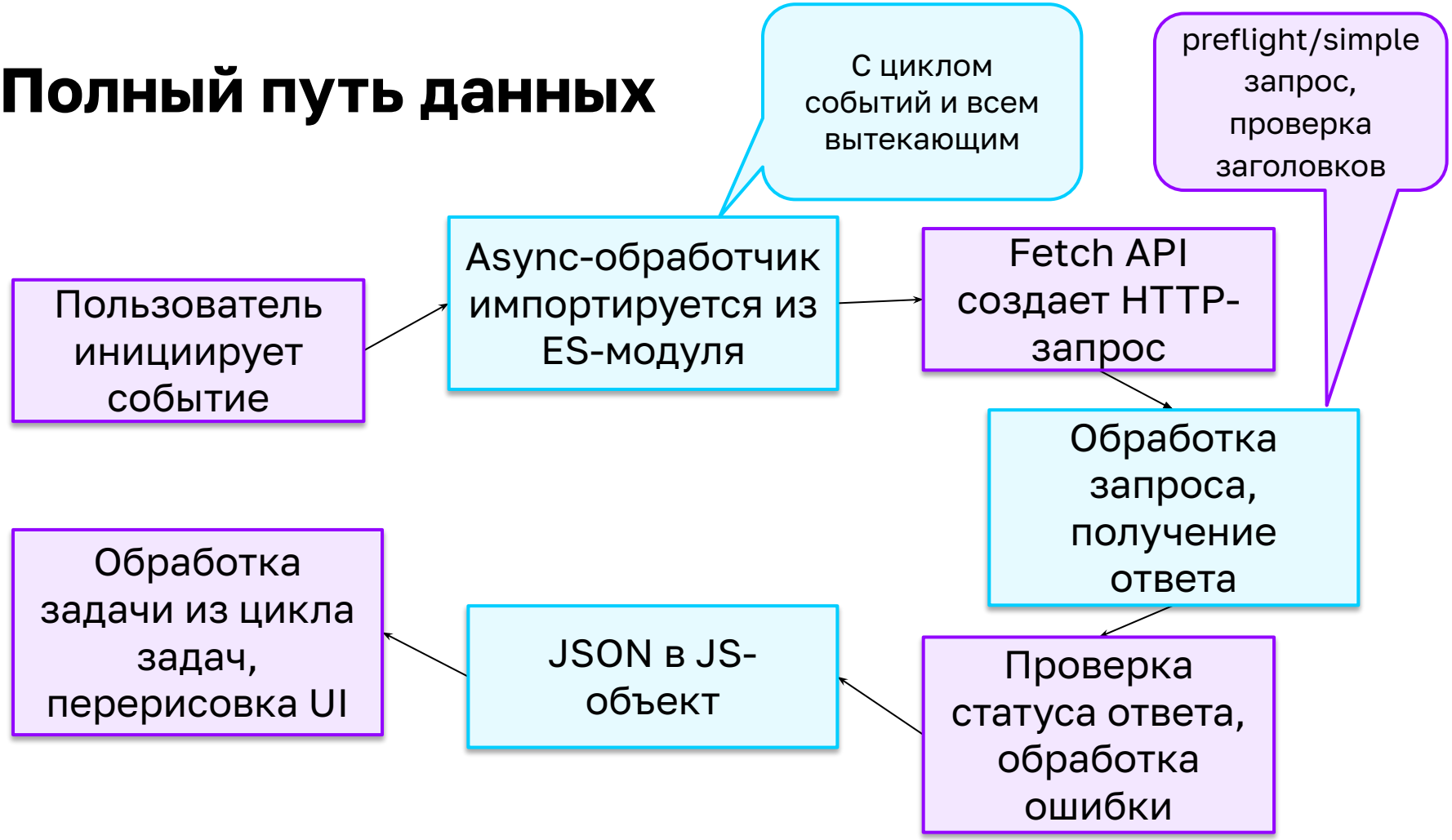
type ReadonlyToDo = Readonly<Todo>;
// {readonly title: string; readonly description: string;}

type PickToDo = Pick<Todo, 'title'>;
// {title: string;}
```

Utility Types - встроенный набор типов-оберток. Используется для упрощения преобразования типов.

- Partial<Type>
- Required<Type>
- Readonly<Type>
- Pick<Types, Keys>

Полный путь данных



Спасибо
за внимание!

ITMO more than a UNIVERSITY